DO NOW — RECALL

Data Structures · Constructors and Encapsulation

Name: _____ Class: _____ Date: _____

1. How do you create an object in C#?

2. What is an instance field?

3. What does modularity improve?

4. What is LIFO?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Use new with a constructor

2. Model answer

Value held separately per object

3. Model answer

Reuse and maintainability

4. Model answer

Last in first out

By the end, you can…

• Initialise objects with constructors

• Explain private fields and public methods

• Implement basic validated access

Vocabulary — know these terms

Recall the term — constructor

Recall the term — encapsulation

Recall the term — private

Recall the term — public

Recall the term — validation

NEW LEARNING

Read, annotate and explain

Build an object with a valid initial state

```
class Account {
private int balance;
public Account(int opening) {
if (opening < 0)
throw new ArgumentOutOfRangeException();
balance = opening;
}
public int GetBalance() => balance;
}
```

Protect the object invariant

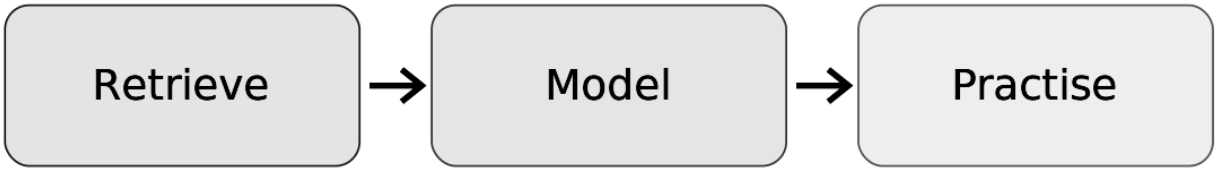
The constructor refuses a negative opening balance before storing it. The field is private, so outside code must use approved public methods instead of bypassing validation.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Build an object with a valid initial state



Three-stage diagram showing retrieve, model, and practise

I DO — WORKED EXAMPLE

Follow the worked method

Construct a valid account; control updates

Constructor sets the initial state

new Account(10) invokes Account(int opening) . It checks 10 is non-negative, then stores 10 in the private field balance .

Method validates the proposed change

savings.Deposit(5) passes the positive-amount check and increases balance from 10 to 15. Outside code cannot change the private field directly.

Check valid and invalid cases

GetBalance() returns 15. Deposit(-3) throws an exception before changing the balance. Encapsulation is controlled access, not merely writing private .

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Trace a different validated account

Using the Account class, run Account current = new Account(20); current.Deposit(7); Then attempt current.Deposit(-2); . What does current.GetBalance() give afterwards if the exception is caught by the caller?

Method / working:

2. Construct a different valid object

A Box has private int width; and a constructor accepting width. Write the validation condition that rejects zero or negative widths, then state which values new Box(4) and new Box(0) should allow.

Method / working:

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Trace a different validated account Using the Account class, run Account current = new Account(20); current.Deposit(7); Then attempt current.Deposit(-2); . What does current.GetBalance()…

DISCUSS — Compare the sequence of steps or states. Identify the first difference in your methods and justify which step is correct.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 2: Construct a different valid object A Box has private int width; and a constructor accepting width. Write the validation condition that rejects zero or negative widths, then state which…

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Apply independently
Write a constructor for class Box that stores integer width. [4]

Question 2

Apply independently
Explain encapsulation. [2]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Apply independently
Suggest validation for a bank deposit method. [2]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

True or false: a constructor has a declared return type void.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

27; invalid deposit rejected

Constructor sets 20; valid deposit makes 27. The second call raises an ArgumentOutOfRangeException before the assignment; when the caller handles it, the balance remains 27 .

IMPROVE: Explain one correction to your own reasoning.

Guided 2 — original CHECK

Apply the same pattern

Identify the precondition before any assignment to width .

Reject invalid initial state

if (w <= 0) throw new ArgumentOutOfRangeException(nameof(w)); width = w; goes inside public Box(int w) . Width 4 is accepted; width 0 is rejected.

IMPROVE: Explain one correction to your own reasoning.